

Lab 2.4: Terraform Modules for Compliance (GCP)

Lab 2.3 built one bucket. This lab builds a pattern. The shift to feel: you don't deploy buckets, you deploy a module that deploys buckets, and the security floor sits inside the module where consumers cannot reach it.

It is on GCP on purpose. Doing the same controls in a second cloud is what proves the control IDs are portable and the resource types are not.

For reading. Code lines wrap to fit the page; copy code from docs/02_04_terraform_modules_for_compliance.md, not the PDF.

This PDF is for reading. Long code lines wrap to fit the page, so copy commands and code from `docs/02_04_terraform_modules_for_compliance.md` in your clone, not from the PDF.

Learning objectives

- Compose a module with a clear interface: inputs, outputs, and hardcoded compliance defaults.
- Encode SC-8, SC-12, SC-13, SC-28, AU-3, AU-11, AC-3, and CM-6 so a consumer cannot switch them off.
- Distinguish a control you **implement** from one you **inherit** from the platform, and express the difference honestly.
- Emit a compliance attestation that Chapter 3 asserts against and Chapter 6 cites.

Controls implemented

Control	Enforced by	Note
SC-12	<code>google_kms_crypto_key.rotation_period</code>	90-day rotation.
SC-13, SC-28	<code>encryption.default_kms_key_name</code> (CMEK)	Your key, not Google's.
AC-3	<code>uniform_bucket_level_access</code> + <code>public_access_prevention</code> = "enforced"	
AU-3	<code>logging.log_bucket</code>	Logs go somewhere other than the bucket being logged.
AU-9	Versioning on the log bucket	
AU-11	<code>retention_policy</code> + <code>lifecycle_rule</code>	Two different mechanisms; see Step 4.
CM-6, CM-8	Merged required labels	Consumers may add, never suppress.
SC-8	Inherited from GCP	See below. This is the interesting one.

The inherited control

On AWS you write a bucket policy denying `aws:SecureTransport = false`, because S3 will happily serve plain HTTP if you let it. On GCP there is no equivalent resource, because the Cloud Storage JSON and XML APIs are TLS-only. The control is satisfied, and you did not implement it.

That distinction has a name in OSCAL: an `implementation-status` of `inherited`, with the provider named as the responsible party. It is not a loophole and it is not a gap. It is the correct answer, and writing "implemented" where you mean "inherited" is the kind of small dishonesty that collapses an assessment when someone asks you to show the code.

Lab 6.1 encodes this. Keep it in mind while you write the module.

Prerequisites

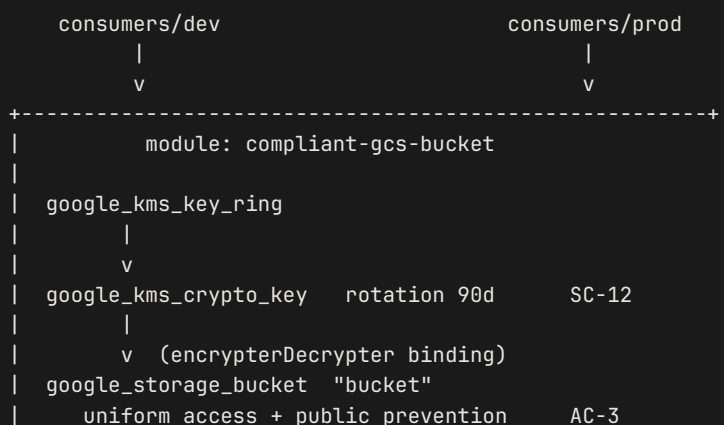
- Run these from inside the devcontainer if you set one up in Lab 0.1: that is where the toolchain and your cloud logins live. `source cgep.env` first, in every new shell.
- **Lab 0.1 Part 2** for the GCP side: creating the project, attaching billing, enabling the APIs, and the two separate authentications. This is the first GCP lab, so do that first if you have only set up AWS so far.
- A GCP project you control with billing enabled. Examples use `your-gcp-project`.
- `gcloud auth login` **and** `gcloud auth application-default login`. The google provider uses ADC.
- Roles: `roles/storage.admin`, `roles/cloudkms.admin`.
- `gcloud services enable cloudkms.googleapis.com`.
- Terraform `>= 1.9`. The cross-variable validation in Step 3 needs it.

Estimated time & cost

- Time: 60 to 75 minutes.
- Cost: KMS key versions bill about **\$0.06 per active version per month**, and 90-day rotation means versions accumulate. Storage is free while the buckets are empty. Destroy same-day and the prorated cost is fractions of a cent.

Read the Cleanup section before you apply. GCP KMS keys cannot be truly deleted, and one setting in this module can make a bucket undestroyable for a year.

Architecture



CMEK	SC-13/SC-28
versioning + retention_policy	AU-11
lifecycle_rule (noncurrent expiry)	AU-11
logging -> log bucket	AU-3
merged required labels	CM-6/CM-8
google_storage_bucket "log"	
versioning	AU-9
uniform access + public prevention	AC-3

+-----+

Two buckets now, not one. The access-log bucket is the v2 addition, and it exists for the same reason it exists in Lab 2.3.

Step-by-step walkthrough

Concept: designing the interface

A module is a directory with an interface. The body decides what is hardcoded; the interface decides what consumers can change. Three rules:

1. `main.tf` hardcodes anything compliance-relevant: encryption, uniform access, versioning, logging, required labels.
2. `variables.tf` exposes only what the business actually changes: project, environment, retention duration, names.
3. `outputs.tf` returns evidence, including a computed attestation.

Notice what the module does not contain: a `provider` block. Providers are configured by the root module and inherited by children. This is the single most important structural difference between this lab and Lab 2.3, and Lab 2.3 gets it wrong on purpose. Lab 2.3's "primitive" declares its own provider, which makes it a root module wearing the word "primitive." The moment you try to call it twice for two regions, that provider block is what stops you.

If you take one thing from this lab into your capstone, take that.

Step 1 `modules/compliant-gcs-bucket/main.tf`

```
# main.tf
terraform {
  required_version = ">= 1.9"
  required_providers {
    google = { source = "hashicorp/google", version = "~> 5.0" }
    random = { source = "hashicorp/random", version = "~> 3.6" }
  }
}
# No provider block. Consumers configure it; this module inherits it.
```

```

}

locals {
  # CM-6 / CM-8: the four required labels. Merged ON TOP of consumer labels,
  # which is the asymmetry that makes them non-negotiable.
  required_labels = {
    project      = var.project_label
    environment   = var.environment
    managed_by    = "terraform"
    compliance_scope = "cge-p-lab"
  }

  effective_labels = merge(var.labels, local.required_labels)

  # GCS bucket names sit in ONE namespace shared by every Google customer, so
  # a fixed name is a name a stranger may already hold. Lab 2.3 met the same
  # wall on S3 and answers it the same way.
  effective_suffix = coalesce(var.bucket_suffix, random_id.bucket_suffix.hex)

  bucket_name = "${var.project_label}-${var.environment}-${var.bucket_name_suffix}-${
local.effective_suffix}"
  log_name     = "${var.project_label}-${var.environment}-${var.bucket_name_suffix}-logs-${
local.effective_suffix}"

  # KMS names are scoped to this project and location, not to Google, so they
  # need no random suffix and deliberately do not carry one. A key ring cannot
  # be deleted from a project, so a name that churns leaves an orphan forever.
  keyring_id = "${var.environment}-${var.bucket_name_suffix}-ring"
  key_id     = "${var.environment}-${var.bucket_name_suffix}-key"
}

resource "random_id" "bucket_suffix" {
  byte_length = 4
}

data "google_storage_project_service_account" "gcs" {
  project = var.gcp_project
}

# SC-12: cryptographic key establishment and management. We own the key.
resource "google_kms_key_ring" "ring" {
  name       = local.keyring_id
  location   = var.kms_location
  project    = var.gcp_project
}

# SC-12 / SC-13: 90-day rotation, expressed in seconds because the API is
# not sorry about it.
resource "google_kms_crypto_key" "key" {
  name           = local.key_id
  key_ring       = google_kms_key_ring.ring.id
  rotation_period = "7776000s" # 90 days

  lifecycle {
    prevent_destroy = false # set true in production
  }
}

```

```

}

resource "google_kms_crypto_key_iam_member" "gcs_encrypter" {
  crypto_key_id = google_kms_crypto_key.key.id
  role          = "roles/cloudkms.cryptoKeyEncrypterDecrypter"
  member        = "serviceAccount:${data.google_storage_project_service_account.gcs.email_address}"
}

# -----
# AU-3 / AU-9: the access-log bucket. Logs belong somewhere other than the
# thing they are logging.
# -----
resource "google_storage_bucket" "log" {
  name      = local.log_name
  project   = var.gcp_project
  location  = var.location

  uniform_bucket_level_access = true
  public_access_prevention    = "enforced"

  # AU-9: audit records must survive overwrite.
  versioning {
    enabled = true
  }

  # AU-11: logs expire on a schedule instead of accumulating forever.
  lifecycle_rule {
    condition {
      age = var.log_retention_days
    }
    action {
      type = "Delete"
    }
  }

  labels = local.effective_labels
}

# GCS writes access logs as the Cloud Storage Analytics group. This binding is
# the GCP analogue of the AWS log-delivery bucket-policy grant in Lab 2.3.
resource "google_storage_bucket_iam_member" "log_writer" {
  bucket = google_storage_bucket.log.name
  role   = "roles/storage.objectCreator"
  member = "group:cloud-storage-analytics@google.com"
}

# -----
# The primary bucket. AC-3 + SC-28 + AU-3 + AU-11 + CM-6 in one declaration.
# -----
resource "google_storage_bucket" "bucket" {
  name      = local.bucket_name
  project   = var.gcp_project
  location  = var.location

  # AC-3: uniform access removes per-object ACLs entirely (the GCP equivalent
  # of BucketOwnerEnforced), and public_access_prevention refuses public

```

```

# grants even if someone with IAM rights tries to add one.
uniform_bucket_level_access = true
public_access_prevention    = "enforced"

versioning {
  enabled = true
}

# SC-13 / SC-28: CMEK. Without this block GCS still encrypts at rest with a
# Google-managed key, which is encryption you cannot rotate, scope, or audit.
encryption {
  default_kms_key_name = google_kms_crypto_key.key.id
}

# AU-11, mechanism one: a retention policy is WORM. Objects cannot be
# deleted until they reach this age. See Step 4 before setting is_locked.
retention_policy {
  retention_period = var.retention_days * 86400
  is_locked       = var.lock_retention_policy
}

# AU-11, mechanism two: lifecycle expires superseded versions. Versioning
# without this is an unbounded bill.
lifecycle_rule {
  condition {
    num_newer_versions = var.keep_noncurrent_versions
    with_state         = "ARCHIVED"
  }
  action {
    type = "Delete"
  }
}

# AU-3: access logs to the dedicated bucket.
logging {
  log_bucket      = google_storage_bucket.log.name
  log_object_prefix = "access-logs/"
}

labels = local.effective_labels

depends_on = [google_kms_crypto_key_iam_member.gcs_encrypter]
}

```

The `depends_on` is genuine. The GCS service account must hold encrypt/decrypt on the key **before** the bucket is created, and there is no data dependency between the bucket and the IAM binding for Terraform to infer it from. Same shape as the legitimate `depends_on` in Lab 2.3.

Why the bucket name has a random tail, and the key ring does not. A GCS bucket name is not unique to your project, it is unique across every Google customer alive. `cgep-lab-dev-data-001` is exactly the name every other student on this lab would pick, and the first one to run it owns it forever. Terraform reports that as `Error 409` partway through an apply, after the key ring and log bucket already exist, which is the worst moment to discover a naming problem.

KMS key rings and keys are scoped to your project and location, so they have no such contest and get no suffix. That is deliberate rather than an oversight: **a key ring cannot be deleted from a Google project**, so a name that changes on every apply leaves a permanent orphan behind each time. Put randomness where the namespace is shared, and nowhere else.

Step 2 The two-mechanism retention lesson

`retention_policy` and `lifecycle_rule` both appear above, and students routinely think one is redundant. They do opposite jobs:

	<code>retention_policy</code>	<code>lifecycle_rule</code>
Effect	Objects cannot be deleted before this age	Objects are deleted at this age
Control	AU-9, AU-11 (preservation)	AU-11 (retention limit), cost
Direction	A floor	A ceiling
Reversible	Only if <code>is_locked = false</code>	Yes

A compliance regime usually wants both: keep for at least N (you cannot destroy evidence early), delete after M (you do not hold data longer than the policy allows). Holding data past its retention limit is itself a finding under most privacy regimes.

One asymmetry worth noticing before you are asked about it. The data bucket sets a CMEK; the log bucket does not, so its objects are encrypted with a Google-managed key. Lab 2.3 did give the S3 log bucket the same CMK, and the mismatch is real. The reason is that GCS delivers usage logs through a Google-owned writer, `cloud-storage-analytics@google.com`, which would need encrypt rights on your key before it could write into a CMEK bucket, and granting a shared Google group access to your key is a trade rather than an obvious win. Both positions are defensible. **What is not defensible is leaving it undecided and unwritten**, which is how an assessor finds it first. If you take this to a capstone, say which you chose and why.

Step 3 `variables.tf` with validation

```
# variables.tf
variable "gcp_project" {
  type      = string
  description = "GCP project ID where the bucket and KMS resources live."
}

variable "location" {
  type      = string
  description = "GCS bucket location. Multi-regions like US and EU are valid."
  default   = "us-central1"
}

variable "kms_location" {
  type      = string
  description = "KMS keyring location. Must be a single region; multi-regions are not supported."
  default   = "us-central1"
}

variable "project_label" {
  type      = string
  description = "Short project identifier."
  validation {
    condition     = can(regex("^[a-z][a-z0-9-]{2,20}$", var.project_label))
    error_message = "project_label must be 3-21 lowercase alphanumerics or hyphens, starting with a letter."
  }
}

variable "environment" {
  type      = string
  description = "Deployment environment."
  validation {
    condition     = contains(["dev", "staging", "prod"], var.environment)
    error_message = "environment must be one of: dev, staging, prod."
  }
}

variable "retention_days" {
  type      = number
  description = "Minimum object retention in days. Production must be >= 365."

  validation {
    condition     = var.retention_days >= 1 && var.retention_days <= 3650
    error_message = "retention_days must be between 1 and 3650."
  }

  validation {
    condition     = var.environment != "prod" || var.retention_days >= 365
    error_message = "retention_days must be >= 365 when environment == \"prod\"."
  }
}

variable "log_retention_days" {
```

```

    type      = number
    description = "AU-11. Age at which access-log objects are deleted."
    default    = 90
  }

  variable "keep_noncurrent_versions" {
    type      = number
    description = "How many superseded versions to retain before lifecycle deletes them."
    default    = 3
  }

  variable "lock_retention_policy" {
    type      = bool
    description = "IRREVERSIBLE. Locking a retention policy prevents shortening or removing it, and the bucket cannot be deleted until every object ages out."
    default    = false
  }

  variable "bucket_name_suffix" {
    type      = string
    description = "Name of this bucket instance, e.g. data-001. Not globally unique on its own; see bucket_suffix."
    validation {
      condition     = can(regex("^[a-z0-9-]{3,30}$", var.bucket_name_suffix))
      error_message = "bucket_name_suffix must be 3-30 lowercase alphanumerics or hyphens."
    }
  }

  # Mirrors var.bucket_suffix in Lab 2.3, same job and same escape hatch: leave
  # it null and the module generates one, set it when a name must stay stable.
  variable "bucket_suffix" {
    type      = string
    default    = null
    description = "Fixed suffix for global uniqueness. Null generates one."

    validation {
      condition     = var.bucket_suffix == null || can(regex("^[a-z0-9-]{3,20}$",
        coalesce(var.bucket_suffix, "gen")))
      error_message = "bucket_suffix must be 3-20 lowercase alphanumerics or hyphens."
    }
  }

  variable "labels" {
    type      = map(string)
    description = "Optional additional labels. Required compliance labels merge on top."
    default    = {}
  }

```

Why two location variables. GCS buckets accept multi-region names like **US** and **EU**. KMS keyrings do not. Setting both from one variable fails with **KMS_RESOURCE_NOT_FOUND_IN_LOCATION** the first time you try **US**. Splitting them keeps the lesson honest.

`lock_retention_policy` is an input, not a hardcoded `false`. An irreversible choice belongs in the consumer's code where a reviewer sees it, rather than buried in a module the consumer never opens. The loud description is part of the control.

Step 4 `outputs.tf` as evidence

```
# outputs.tf
output "bucket_url" {
  value      = google_storage_bucket.bucket.url
  description = "gs:// URL of the compliant bucket."
}

output "bucket_name" {
  value      = google_storage_bucket.bucket.name
  description = "Name of the compliant bucket."
}

output "log_bucket_name" {
  value      = google_storage_bucket.log.name
  description = "Name of the access-log bucket."
}

output "kms_key_id" {
  value      = google_kms_crypto_key.key.id
  description = "Resource ID of the CMEK protecting this bucket."
}

output "compliance_attestation" {
  description = "Computed attestation of the controls this module enforces."
  value = {
    # Read back from the resource, never from the variable that set it. An
    # attestation that repeats its own input cannot be wrong, which means it
    # cannot be right either: delete the encryption block and a hardcoded
    # "cmek" still says cmek.
    encryption_type      = length(google_storage_bucket.bucket.encryption) > 0 ? "cmek" : "google-
managed"
    kms_key_id           = one([for e in google_storage_bucket.bucket.encryption :
e.default_kms_key_name])
    kms_rotation_period  = google_kms_crypto_key.key.rotation_period
    versioning_enabled   = google_storage_bucket.bucket.versioning[0].enabled
    log_versioning_enabled = google_storage_bucket.log.versioning[0].enabled
    public_access_prevention = google_storage_bucket.bucket.public_access_prevention
    uniform_access_enforced = google_storage_bucket.bucket.uniform_bucket_level_access
    access_logging_target = google_storage_bucket.bucket.logging[0].log_bucket
    # is_locked is the sharp case. Locking is a one-way API call that can be
    # made from the console without Terraform, so var.lock_retention_policy
    # would attest false on a bucket that is genuinely, permanently locked.
    retention_period_days = one([for r in google_storage_bucket.bucket.retention_policy :
r.retention_period / 86400])
    retention_policy_locked = one([for r in google_storage_bucket.bucket.retention_policy :
r.is_locked])
  }
```

```

# condition and action are SETS, not lists, so [0] is a type error that
# terraform validate accepts and terraform plan rejects. Lab 2.3's own
# note on this applies verbatim: one() raises where tolist(...)[0] would
# silently return an arbitrary element.
log_retention_days = one([
  for r in google_storage_bucket.log.lifecycle_rule :
    one(r.condition).age
  if one(r.action).type == "Delete"
])

required_labels_present = alltrue([
  for k in keys(local.required_labels) :
    contains(keys(google_storage_bucket.bucket.labels), k)
])

# SC-8 is satisfied by the platform, not by this module. Saying so in the
# attestation is the honest form; Lab 6.1 maps this to an OSCAL
# implementation-status of "inherited".
transit_encryption = "inherited-gcp-tls-only"
}
}

```

Compare this shape to Lab 2.3's `compliance_attestation`. Different cloud, different resource types, same evidence surface. That similarity is what lets one Rego library and one OSCAL component describe both.

Step 5 Consumer: dev

```

# consumers/dev/main.tf
terraform {
  required_version = ">= 1.9"
  required_providers {
    google = { source = "hashicorp/google", version = "~> 5.0" }
  }
}

provider "google" {
  project = var.gcp_project
  region  = "us-central1"
}

variable "gcp_project" { type = string }

module "data_bucket" {
  source = "../../modules/compliant-gcs-bucket"

  gcp_project      = var.gcp_project
  project_label    = "cgep-lab"
  environment      = "dev"
  retention_days   = 30
  bucket_name_suffix = "data-001"
}

```

```

output "compliance_attestation" { value = module.data_bucket.compliance_attestation }
output "bucket_name" { value = module.data_bucket.bucket_name }
output "log_bucket_name" { value = module.data_bucket.log_bucket_name }
output "bucket_url" { value = module.data_bucket.bucket_url }

```

Six lines of business configuration. Twenty-plus controls. The provider lives here, in the root module, exactly where it belongs.

Step 6 Consumer: prod

Same module, two values changed:

```

module "data_bucket" {
  source = "../../modules/compliant-gcs-bucket"

  gcp_project      = var.gcp_project
  project_label    = "cgep-lab"
  environment      = "prod"
  retention_days   = 365
  bucket_name_suffix = "data-001"
}

```

Step 7 Apply dev

Plan prod, but do not apply it. A 365-day retention takes a year to expire.

```

cd consumers/dev
terraform init
terraform fmt && terraform validate
terraform plan -out=tfplan
terraform apply -auto-approve tfplan

```

Tail:

```

compliance_attestation = {
  "access_logging_target" = "cgep-lab-dev-data-001-logs-9f2ac41b"
  "encryption_type"       = "cmek"
  "kms_rotation_period"   = "7776000s"
  "log_retention_days"    = 90
  "log_versioning_enabled" = true
  "public_access_prevention" = "enforced"
  "required_labels_present" = true
  "retention_period_days"  = 30
  "retention_policy_locked" = false
  "transit_encryption"     = "inherited-gcp-tls-only"
  "uniform_access_enforced" = true
  "versioning_enabled"     = true
}

```

That output is your SC-12 / SC-13 / SC-28 / AC-3 / AU-3 / AU-9 / AU-11 / CM-6 attestation in machine-readable form.

Step 8 The negative test

Copy `consumers/dev` to `consumers/negative-test`, set `environment = "prod"` and leave `retention_days = 30`:

```
Plan: 6 to add, 0 to change, 0 to destroy.

Error: Invalid value for variable

  on main.tf line 29, in module "data_bucket":
  29:   retention_days      = 30

   var.environment is "prod"
   var.retention_days is 30

retention_days must be >= 365 when environment == "prod".

This was checked by the validation rule at
../modules/compliant-gcs-bucket/variables.tf:54,3-13.
```

This is the lesson. The compliance check ran at `terraform plan`, before any resource existed, with a message specific enough that the developer fixes it without filing a ticket. Preventive beats detective, and it is cheaper.

Two things about that output that surprise people.

Terraform renders the whole plan first, then fails. You will see six resources listed under `+ create` and a line reading `Plan: 6 to add` above the error. Nothing was created. A plan is a proposal, and this one was refused before anything acted on it, so the `negative-test.txt` in your evidence folder is a plan that never ran. Say that in your write-up, because a reviewer skimming for `6 to add` will otherwise read it as a near miss.

`terraform validate` says Success! on this exact configuration. Both values are literals sitting in `main.tf`, so it looks like something a static check should catch, and it is not: `validate` checks syntax, types and references, never values. Variable validation rules are evaluated during `plan`. The practical consequence is worth carrying into Lab 4.3: **a pipeline whose only Terraform gate is `terraform validate` passes this configuration.** The pipeline you build there runs `validate` and then `plan` under `set -euo pipefail`, which is why it catches what validate alone would wave through.

Run a second negative test: try to suppress a required label.

```
labels = { compliance_scope = "not-in-scope" }
```

Plan it. The label comes back as `cge-p-lab`, because `merge(var.labels, local.required_labels)` puts the required set last and last wins. **The consumer can add labels and cannot override the compliance ones.** That asymmetry is the entire design, and watching it refuse is better than reading about it.

Verification

gcloud silently drops field names it does not recognize. The projection in `--format` is a filter, not a query: ask for a field that does not exist and gcloud prints the fields it did understand and says nothing at all about the one it did not. An earlier draft of this section asked for `logging`, whose real name is `logging_config`, and the output simply had no logging section in it.

Read what that means for evidence. **A control that is genuinely missing and a field name you got wrong produce identical output.** If you are verifying a control and its section is absent, your first move is to confirm the field name, not to conclude the control failed. The field names come from gcloud's own resource model rather than from the JSON API, so they are `logging_config` and `default_kms_key` even though the REST API calls them `logging` and `encryption.defaultKmsKeyName`.

That trap is exactly why this section asserts rather than prints. Run it from `consumers/dev`:

```
bash scripts/verify.sh consumers/dev
```

Every line names a control and either passes or fails with what it expected and what the project actually returned. The run ends in `VERIFIED` or `NOT VERIFIED` and sets the exit status accordingly.

On the one enforcement check. It is a read, not a write, and that is deliberate. The obvious way to prove public access prevention works is to try adding `allUsers` as a viewer and watch it be refused. If the control were missing, that test would succeed and leave your bucket public. A check that damages the thing it is checking when it fails is not worth running, so this one asks for the bucket listing anonymously and requires a 401 or 403.

The same reasoning rules out watching the retention policy deny a delete. Doing that means putting an object in the bucket, and a 30 day retention floor then prevents you from deleting it, which prevents you from destroying the bucket, for 30 days. The configuration assertion is the honest trade here, and the cleanup section explains the rest.

```
#!/usr/bin/env bash
# scripts/verify.sh
# Verification for Lab 2.4. Every check asserts a value and exits non-zero if
# the project disagrees.
#
# Run it from consumers/dev, where the outputs live.
#
# One trap this replaces: gcloud's --format projection is a filter, not a
# query. Ask for a field it does not recognize and it prints the fields it did
# understand and says nothing about the one it did not, exit code 0. An earlier
# draft asked for `logging` instead of `logging_config` and simply had no
# logging section in its output. A control that is genuinely missing and a
# field name you typed wrong look identical.
set -uo pipefail
```

```

# Point this at the workspace whose `terraform output` describes the resources
# you want checked, so it does not matter where you run it from. Guessing the
# working directory is how a verification script cheerfully checks the wrong
# account and passes.
WORKSPACE="${1:-.}"
cd "$WORKSPACE" 2>/dev/null || { echo "no such workspace: $WORKSPACE" >&2; exit 1; }

FAILED=0

pass() { printf ' PASS %-8s %s\n' "$1" "$2"; }
fail() { printf ' FAIL %-8s %s\n' "$1" "$2" >&2; FAILED=1; }

check() { # check CONTROL LABEL EXPECTED ACTUAL
  if [[ "$3" == "$4" ]]; then
    pass "$1" "$2 is $4"
  else
    fail "$1" "$2: expected '$3', got '${4:- (nothing)}'"
  fi
}

BUCKET=$(terraform output -raw bucket_name)
LOGS=$(terraform output -raw log_bucket_name)
ATT=$(terraform output -json compliance_attestation)
KMS_ID=$(jq -r '.kms_key_id' <<<"$ATT")

# projects/P/locations/L/keyRings/R/cryptoKeys/K
KEY_PROJECT=$(awk -F/ '{print $2}' <<<"$KMS_ID")
KEY_LOCATION=$(awk -F/ '{print $4}' <<<"$KMS_ID")
KEY_RING=$(awk -F/ '{print $6}' <<<"$KMS_ID")
KEY_NAME=$(awk -F/ '{print $8}' <<<"$KMS_ID")

bucket_field() { # bucket_field BUCKET FIELD
  gcloud storage buckets describe "gs://$1" --format="value($2)" 2>/dev/null
}

# Booleans are read as JSON rather than through value(), which renders them
# with Python's capitalisation. Asserting against "True" when the tool happens
# to emit "true" is a failure that teaches nothing.
bucket_json() { # bucket_json BUCKET JQ_PATH
  gcloud storage buckets describe "gs://$1" --format=json 2>/dev/null \
    | jq -r "$2 // empty"
}

echo "=== configuration ==="

# SC-13 / SC-28. The key itself, not merely that some key is set.
check SC-28 "default CMEK" "$KMS_ID" "$(bucket_field "$BUCKET" default_kms_key)"

# SC-12. Ninety days, in seconds, because the API is not sorry about it.
check SC-12 "key rotation" "7776000s" \
  "$(gcloud kms keys describe "$KEY_NAME" --keyring="$KEY_RING" \
    --location="$KEY_LOCATION" --project="$KEY_PROJECT" \
    --format="value(rotationPeriod)" 2>/dev/null)"

# CP-9 and AU-9.

```



```

check CP-9 "data versioning" "true" "${bucket_json "$BUCKET" .versioning_enabled}"
check AU-9 "log versioning" "true" "${bucket_json "$LOGS" .versioning_enabled}"

# AC-3 and AC-6.
check AC-3 "public access prevention" "enforced" \
  "${bucket_field "$BUCKET" public_access_prevention}"
check AC-6 "uniform bucket access" "true" \
  "${bucket_json "$BUCKET" .uniform_bucket_level_access}"

# AU-3. The target has to be the log bucket, not merely some bucket. This is
# the field that was silently absent when the name was wrong.
check AU-3 "log target" "$LOGS" "${bucket_field "$BUCKET" logging_config.logBucket}"

# AU-11. Asserted against the module's own attestation, so a disagreement
# between the code and the project shows up as a failure rather than as two
# numbers nobody compared.
check AU-11 "retention seconds" \
  "$(( $(jq -r '.retention_period_days' <<<"$ATT") * 86400 ))" \
  "${bucket_field "$BUCKET" retention_policy.retentionPeriod}"

echo "=== enforcement ==="

# AC-3, without touching anything. An unauthenticated read of the bucket
# listing must be refused. This is deliberately a read: the obvious GCP denial
# test is to try adding allUsers as a viewer, and if the control were missing
# that test would succeed and leave your bucket public. A check that damages
# the thing it is checking when it fails is not worth running.
CODE=$(curl -s -o /dev/null -w '%{http_code}' \
  "https://storage.googleapis.com/storage/v1/b/${BUCKET}/o" 2>/dev/null)
if [[ "$CODE" == "401" || "$CODE" == "403" ]]; then
  pass "AC-3" "anonymous listing refused with HTTP $CODE"
else
  fail "AC-3" "anonymous listing returned HTTP ${CODE:-no response}, expected 401 or 403"
fi

echo
if [[ $FAILED -eq 0 ]]; then
  echo "VERIFIED: every control asserted above holds in the project."
else
  echo "NOT VERIFIED: at least one control does not hold. Do not capture evidence yet." >&2
fi
exit $FAILED

```

Capture the evidence the checklist asks for

```

# evidence/ lives at the repository root, not in the workspace you are in
EVIDENCE=$(git rev-parse --show-toplevel)/evidence/lab-2-4"
mkdir -p "$EVIDENCE"
terraform show -json tfplan > "$EVIDENCE/plan.json"
terraform output -json compliance_attestation > "$EVIDENCE/attestation.json"

```

The negative test is the one worth keeping, because it is the proof the module refuses rather than merely defaults:

```
( cd ../negative-test && terraform init -input=false >/dev/null && terraform plan 2>&1 ) \  
| tee "$EVIDENCE/negative-test.txt"
```

The subshell is deliberate: the `cd` ends with it, so the `tee` path stays relative to where you started and you are not left in a different directory. `negative-test` is a workspace you have never initialized, so it needs its own `terraform init` before it can plan.

Portfolio submission checklist

- ☐ `terraform/modules/compliant-gcs-bucket/{main.tf,variables.tf,outputs.tf,README.md}`
- ☐ `terraform/consumers/{dev,prod}/` committed.
- ☐ Module README lists each control by family **and marks SC-8 as inherited, with the reason.**
- ☐ `evidence/lab-2-4/plan.json`
- ☐ `evidence/lab-2-4/attestation.json` from `terraform output -json compliance_attestation`
- ☐ `evidence/lab-2-4/negative-test.txt` capturing both refusals: the prod retention rule and the label-override attempt.

Troubleshooting

- `KMS_RESOURCE_NOT_FOUND_IN_LOCATION` when `location` is `US`. Keyrings need a single region. The two-variable split is the fix.
- `Permission cloudkms.cryptoKeyEncrypterDecrypter denied` during bucket creation. The GCS service agent needs encrypt/decrypt on the key before the bucket exists. The `depends_on` sequences it; keep it if you refactor.
- **Access logs never appear.** GCS access logs are delivered roughly hourly, and only when there is traffic. Confirm the `cloud-storage-analytics@google.com` binding exists on the log bucket.
- **Retention policy cannot be shortened.** It can be lengthened or removed only while `is_locked = false`. Once locked, neither.
- `googleapi: Error 409: The requested bucket name is not available`. GCS names are global, so this used to be routine. The module now generates a random suffix, so a 409 means you pinned `bucket_suffix` to a value someone else holds. Unset it and let the module choose.
- **reauth related error (invalid_rapt).** Run `gcloud auth application-default login` again. ADC tokens expire and Terraform will not refresh them.

Cleanup

```
cd consumers/dev
terraform destroy -auto-approve
```

Three things that will surprise you:

1. **A locked retention policy makes the bucket undestroyable** until every object ages out. With `retention_days = 365` and `lock_retention_policy = true`, that is a year. This is why the variable defaults to `false` and says so loudly.
2. **KMS crypto keys are never truly deleted.** Destroying a key version enters a 30-day soft-delete. The key and keyring objects remain indefinitely; keyrings cannot be deleted at all. They are free, so this is tidiness rather than cost, but a `terraform destroy` that reports success has not removed them.
3. **The log bucket may hold objects** and refuse to destroy. Empty it first if the bucket saw traffic.

How this feeds the capstone

- **The module discipline is the deliverable**, more than the module. Your capstone's KMS and S3 hardening should be a module both dev and prod consume, so the floor is enforced once.
- **Ch 3**, Rego reads `compliance_attestation` and refuses to merge a plan that cannot produce it.
- **Ch 6**, the OSCAL component for `compliant-gcs-bucket-v1` cites this module path as its implementation and the attestation JSON as evidence. The `transit_encryption` field becomes an `inherited` implementation status, which is the honest OSCAL and the thing most candidates get wrong.

Revision history

v2 (current)

- Added an access-log bucket, with versioning and a lifecycle rule, adding AU-3 and AU-9. The module previously logged nowhere.
- `lock_retention_policy` exposed as an input rather than hardcoded, so an irreversible choice appears in the consumer's code.
- Added a second lifecycle mechanism for superseded versions, and documented why `retention_policy` (a floor) and `lifecycle_rule` (a ceiling) are both needed.
- The attestation now records `transit_encryption` as inherited from the platform, rather than leaving SC-8 unstated.
- Added a second negative test showing that a consumer cannot suppress a required label.

v1

Initial release: single bucket, no access logging, `is_locked` hardcoded.